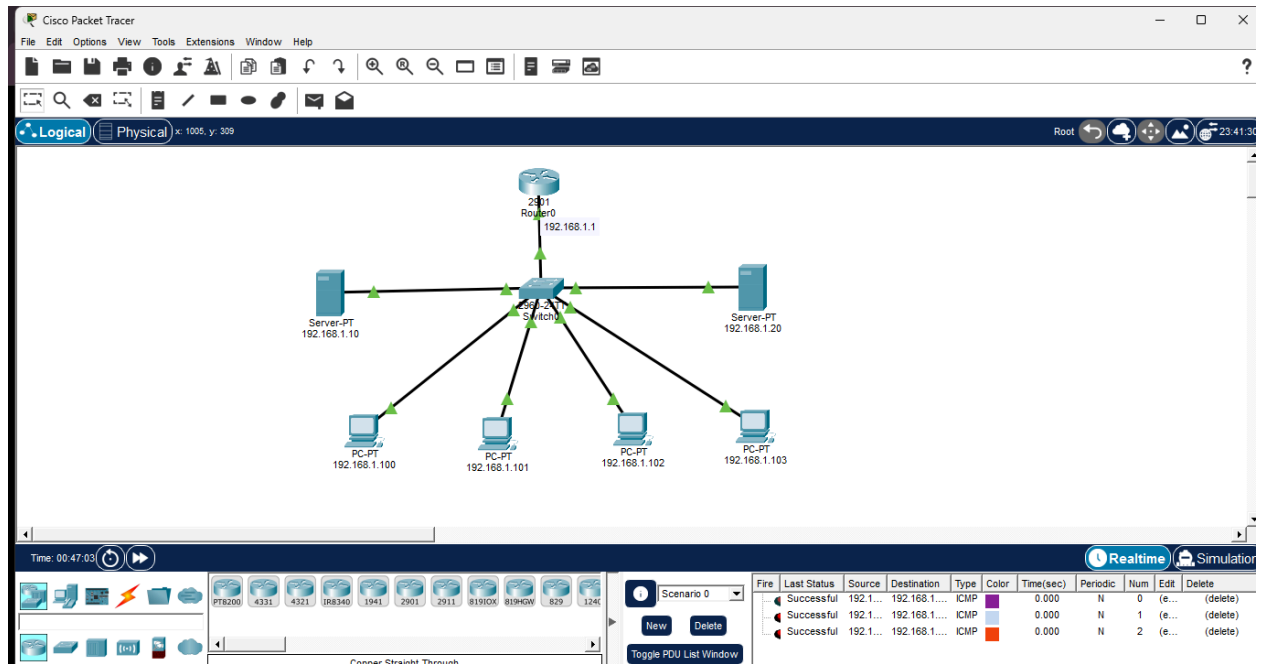


SET 4

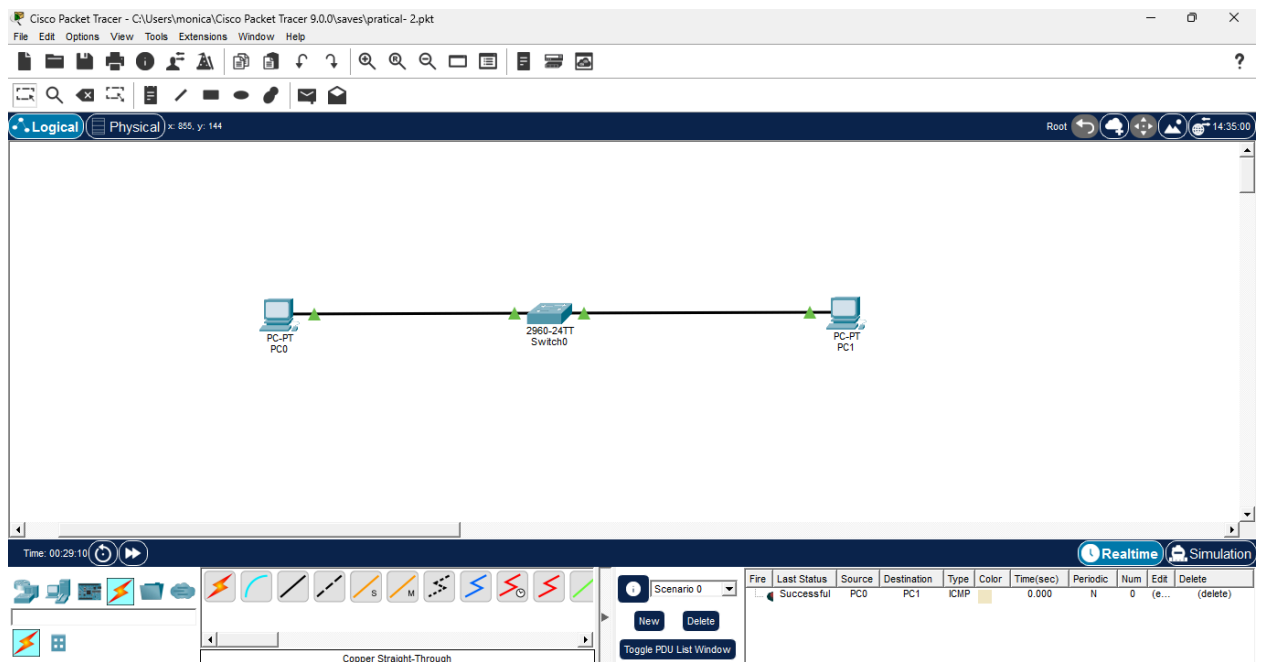
1. Assume that you are the network administrator of a data center requiring load balancing across 2 servers. Design and configure the network using Packet Tracer to demonstrate basic load distribution.
2. Configure Port Security on a switch in Packet Tracer to allow only specific MAC addresses to connect to a port, and verify that unauthorized devices are blocked.
3. Using Wireshark, Capture and analyze the packets generated during a Traceroute operation (ICMP TTL Exceeded) for different header fields and payload of the concerned packet.
4. Using C program, Implement and demonstrate parity bit generation and checking for error detection on a given data.

MODEL PRACTICAL EXAM

QUESTION 1:



QUESTION - 2



QUESTION 3:

Capturing from Wi-Fi						
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
icmp						
No.	Time	Source	Destination	Protocol	Length	Info
315	9.731521500	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=252/64512, ttl=1 (no response found!)
328	9.896158600	172.25.96.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
329	9.898400300	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=253/64768, ttl=1 (no response found!)
330	9.901978400	172.25.96.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
331	9.903466400	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=254/65024, ttl=1 (no response found!)
332	10.071590000	172.25.96.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
363	10.749854200	172.25.96.1	172.25.102.49	ICMP	70	Destination unreachable (Port unreachable)
417	12.001862000	172.25.96.1	172.25.102.49	ICMP	70	Destination unreachable (Port unreachable)
520	15.919360600	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=255/65280, ttl=2 (no response found!)
521	15.934461600	172.22.10.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
522	15.936078600	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=256/1, ttl=2 (no response found!)
523	16.148750600	172.22.10.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
524	16.150061800	172.25.102.49	8.8.8.8	ICMP	106	Echo (ping) request id=0x0001, seq=257/257, ttl=2 (no response found!)
525	16.169725000	172.22.10.1	172.25.102.49	ICMP	70	Time-to-live exceeded (Time to live exceeded in transit)
529	16.211782800	172.22.10.1	172.25.102.49	ICMP	70	Destination unreachable (Port unreachable)

> Frame 315: Packet, 106 bytes on wire (848 bits), 106 bytes captured (848 bits) on interface \Device\NPF{...}	0000	24 d5 e4 6d c0 e7 8c 8b 5b 88 4d 29 08 00 45 00	\$-m-...[M]-E:
> Ethernet II, Src: WNC_88:4d:29 (8c:8b:5b:88:4d:29), Dst: Cisco_6d:c0:e7 (24:d5:e4:6d:c0:e7)	0010	00 5c 21 fb 00 00 01 01 75 4c ac 19 66 31 08 08	..I.....uL..f..
> Internet Protocol Version 4, Src: 172.25.102.49, Dst: 8.8.8.8	0020	08 08 08 00 f7 02 00 01 00 fc 00 00 00 00 00 00	
> Internet Control Message Protocol	0030	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0040	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0050	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
	0060	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

```

Microsoft Windows [Version 10.0.26200.9168]
(c) Microsoft Corporation. All rights reserved.

C:\Users\monica>tracert 8.8.8.8

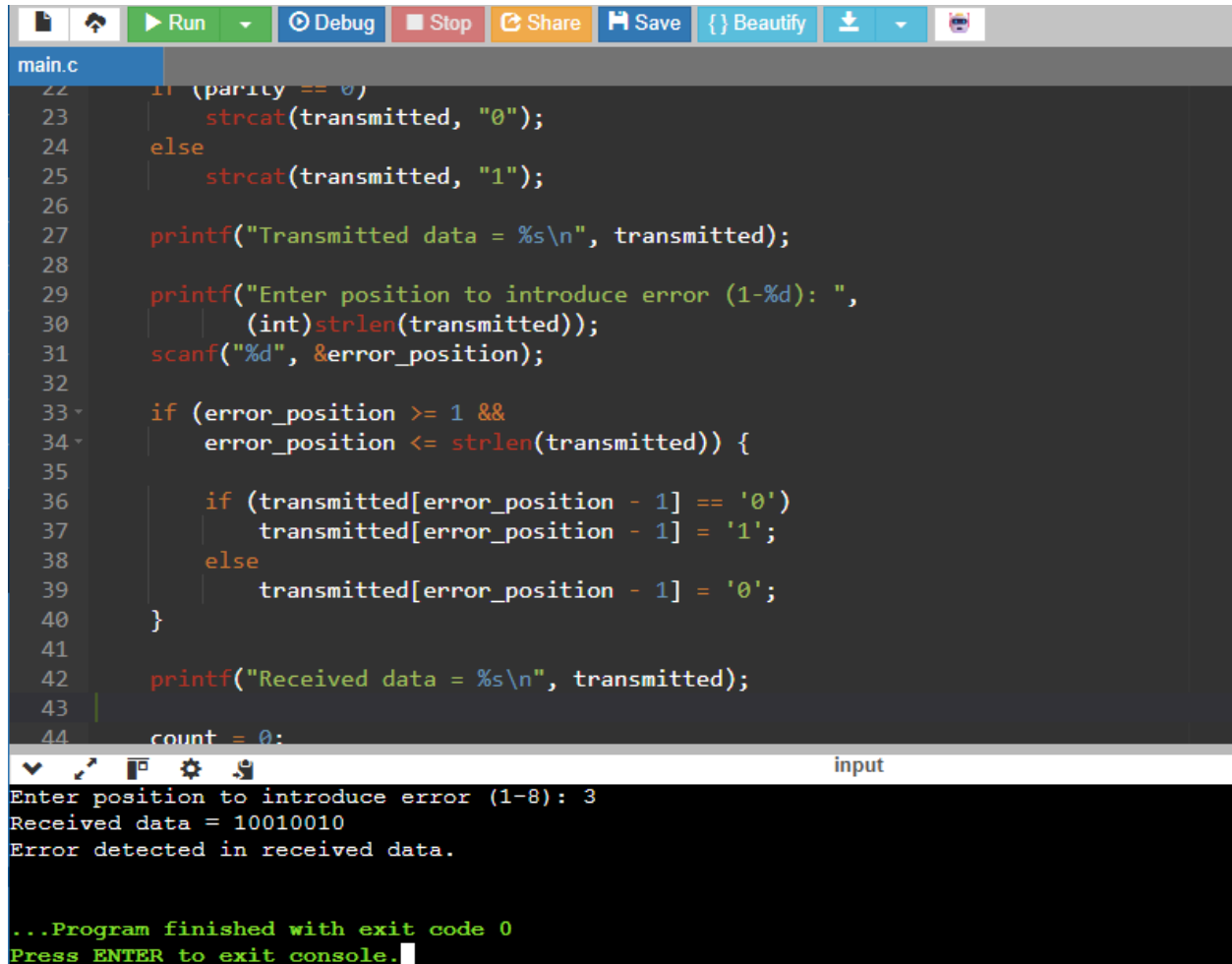
Tracing route to dns.google [8.8.8.8]
over a maximum of 30 hops:
  0  164 ms  3 ms  168 ms  172.25.96.1
  1  15 ms  212 ms  19 ms  172.22.10.1
  2  24 ms  49 ms  6 ms  10.139.187.225
  3  * * * Request timed out.
  4  * * * Request timed out.
  5  * * * Request timed out.
  6  * * * Request timed out.
  7  * * * Request timed out.
  8  86 ms  9 ms  20 ms  72.14.213.20
  9  48 ms  100 ms  26 ms  216.239.43.133
 10 133 ms  37 ms  113 ms  142.251.55.43
 11 246 ms  26 ms  25 ms  dns.google [8.8.8.8]

Trace complete.

C:\Users\monica>

```

QUESTION 4:



The image shows a code editor window with a file named `main.c`. The code is a C program that simulates a data transmission with a parity bit and an error detection mechanism. The code is as follows:

```
22 if (parity == 0)
23     strcat(transmitted, "0");
24 else
25     strcat(transmitted, "1");
26
27 printf("Transmitted data = %s\n", transmitted);
28
29 printf("Enter position to introduce error (1-%d): ",
30       (int)strlen(transmitted));
31 scanf("%d", &error_position);
32
33 if (error_position >= 1 &&
34     error_position <= strlen(transmitted)) {
35
36     if (transmitted[error_position - 1] == '0')
37         transmitted[error_position - 1] = '1';
38     else
39         transmitted[error_position - 1] = '0';
40 }
41
42 printf("Received data = %s\n", transmitted);
43
44 count = 0;
```

The console output shows the program's execution:

```
Enter position to introduce error (1-8): 3
Received data = 10010010
Error detected in received data.

...Program finished with exit code 0
Press ENTER to exit console.
```

1. Load Balancing across 2 servers Using Packet Tracer

Aim:

To design and configure a network using Cisco Packet Tracer to demonstrate basic load distribution across two servers.

Procedure

1. Open Cisco Packet Tracer.

2. Add:

* 1 Router

* 1 Switch

* 2 Servers

* 2 PCs

3. Connect the PCs and Servers to the Switch using copper straight-through cables.

4. Connect the switch to the router

5. Configure the IP addresses:

Device	IP Address	Subnet Mask
Router	192.168.1.1	255.255.255.0
Server1	192.168.1.10	255.255.255.0

Server 2

192.168.1.20

255.255.255.0

PC1

192.168.1.30

255.255.255.0

PC2

192.168.1.40

255.255.255.0

6. Enable the required service, such as HTTP, on both servers.

7. Configure the network gateway as 192.168.1.1.

8. Test connectivity using the ping command.

9. Send requests from the client PCs to the two servers.

10. Demonstrate basic load distribution by directing different client requests to server 1 and server 2.

Sample Output

PC1 > ping 192.168.1.10

Reply from 192.168.1.10: bytes=32 time<1ms

Reply from 192.168.1.10: bytes=32 time<1ms

PC2 > ping 192.168.1.20

Reply from 192.168.1.20: bytes=32 time<1ms

Reply from 192.168.1.20: bytes=32 time<1ms

Result

The network was successfully designed and configured in Packet Tracer, and basic load distribution across two servers was demonstrated successfully.

2.

2. Port Security Using Packet Tracer

Aim

To configure Port Security on a switch to allow only specific MAC addresses and block unauthorized devices.

Procedure

1. Open CISCO Packet Tracer

2. Add:

* 1 Switch

* 2 PCs

3. Connect both PCs to the Switch

4. Check the MAC address of the authorized PC

5. Open the Switch CLI

6. Enter the following commands:

Switch > enable

Switch# configure terminal

Switch(config)# switchport mode access

Switch(config)# switchport port-security

7. Connect the authorized PC to Fa 0/1.

8. Generate traffic using ping.

9. Replace the authorized PC with another unauthorized PC.
10. Generate traffic again
11. The Switch detects the unauthorized MAC address and places the port into a security-violation state.

Sample Output

For checking Port Security:

```
Switch# Show port-security interface  
fast Ethernet 0/1
```

Port Security : Enabled

Port Status : Secure-up

Maximum MAC addresses : 1

Total MAC addresses : 1

Security Violation Count : 0

Result:

Port Security was successfully configured and the unauthorized device was blocked from accessing the switch port.

3. Traceroute Packet Analysis Using Wireshark.

Aim

To capture and analyze packets generated during a Traceroute operation using Wireshark, particularly ICMP TTL Exceeded Packets and study their header fields and payload.

Procedure

1. Install and open Wireshark
2. Select the active network interface
3. Start packet capture.
4. Open Command Prompt / Terminal

5. Run

Windows: `tracert google.com`

Linux: `traceroute google.com`

Linux:

`traceroute google.com`

6. Observe the packets captured by Wireshark.

7. Apply the display filter.

`icmp`

8. Select an ICMP Time-to-live exceeded Packet

9. Analyze

10. Stop the capture and record the observation.

Sample Output

Traceroute:

Tracing route to google.com

1	41 ms	21ms	21ms	192.168.1.1
2	5ms	6ms	5ms	10.10.1.1
3	15ms	14ms	16ms	

Wireshark Observation

Protocol : ICMP

ICMP Type : 11

ICMP Code : 0

Message : Time-to-live exceeded

TTL : 64

Source IP : 192.168.1.1

Destination IP: Client IP

Important ICMP values

Field	Value
-------	-------

ICMP	11
------	----

ICMP Code	0
-----------	---

Meaning	Time-to live exceeded
---------	-----------------------

Protocol	ICMP
----------	------

Result

Traceroute packets were successfully captured using Wireshark, and the ICMP TTL Exceeded Packet headers and payload were analyzed successfully.

4. Parity Bit Generation and Error checking using C

Aim:-

To implement parity bit generation and checking using a C Program for detecting errors in transmitted data.

Procedure

1. Start the C program.
2. Read the binary data from the user.
3. Count the number of 1s in the data.
4. For even parity, generate:
 - * Parity bit 0 if number of 1s is even
 - * Parity bit 1 if number of 1s is odd.
5. Append the parity bit to the data
6. Read the received data.
7. Count the number of 1s again
8. Check whether the total number of 1s is even
9. If even $\rightarrow$ No error detected
10. If odd $\rightarrow$ Error detected.

C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char data[100];
```

```
    int i, Count = 0;
```

```
    Print ("Enter binary data:");
```

```
    Scanf ("%s", data);
```

```
    for (i = 0; i < strlen (data); i++)
```

```
    { if (data [i] == '1')
```

```
        Count ++;
```

```
    }
```

```
    if (Count % 2 == 0)
```

```
        Printf ("Even parity bit = 0\n");
```

```
    else
```

```
        Print ("Even parity bit = 1\n");
```

```
    Print ("Parity checking completed.\n");
```

```
    return;
```

```
}
```

sample output

Input:

Enter binary data: 1011001

Number of 1s = 4

Even parity bit = 0

Parity checking completed

Result

The C program was successfully implemented to generate and check the Parity bit and parity-based error detection was demonstrated successfully.